Marwadi University Marwadi Chandarana Group	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: DSIP	**Aim:**	
Experiment No:	**Date:**	**Enrolment No: 92301733047**

Task 1: Image Representation and Signal Analysis

1. Continuous and Discrete Time Signals:

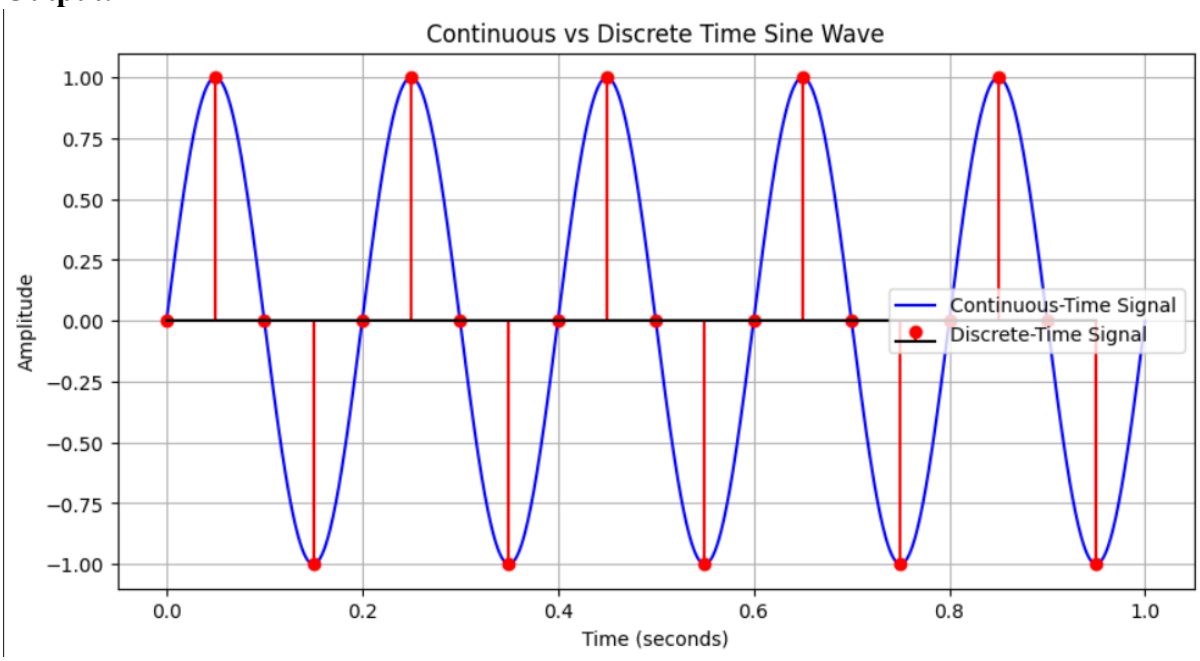
Part A: Write a Python function to generate a continuous-time signal (e.g., a sine wave) and discretize it to create a discrete-time signal. Plot both signals. [CO1, CO2]

Code:

```
import numpy as np
import matplotlib.pyplot as plt
# Part A: Continuous and Discrete Time Signals
def continuous_and_discrete_signal():
    # Continuous time signal parameters
    f = 5  # frequency in Hz
    t_cont = np.linspace(0, 1, 1000)  # continuous time (fine resolution)
    x_cont = np.sin(2 * np.pi * f * t_cont)  # continuous sine wave

    # Discretization parameters
    fs = 20  # sampling frequency (samples per second)
    t_disc = np.arange(0, 1, 1/fs)  # discrete time
    x_disc = np.sin(2 * np.pi * f * t_disc)  # discrete sine wave

    # Plotting
    plt.figure(figsize=(10, 5))

    # Continuous-time signal
    plt.plot(t_cont, x_cont, label='Continuous-Time Signal', color='blue')

    # Discrete-time signal
    plt.stem(t_disc, x_disc, linefmt='r-', markerfmt='ro', basefmt='k-', label='Discrete-
Time Signal')

    plt.title('Continuous vs Discrete Time Sine Wave')
    plt.xlabel('Time (seconds)')
    plt.ylabel('Amplitude')
    plt.legend()
    plt.grid(True)
    plt.show()
```

```
# Call the function
continuous_and_discrete_signal()
```

Output:



Part B: Create a case study where you discuss the representation of a 2D image as a discrete-time signal. Use a sample image and convert it into a 2D array of pixel values. [CO1, CO2]

Answer:

- A 2D image can be viewed as a two-dimensional discrete-time signal.
- Each pixel corresponds to a discrete sample in space — i.e., the signal is defined only at integer coordinates (m, n).
- For a grayscale image:
  f(m,n)=pixel intensity at position (m,n)

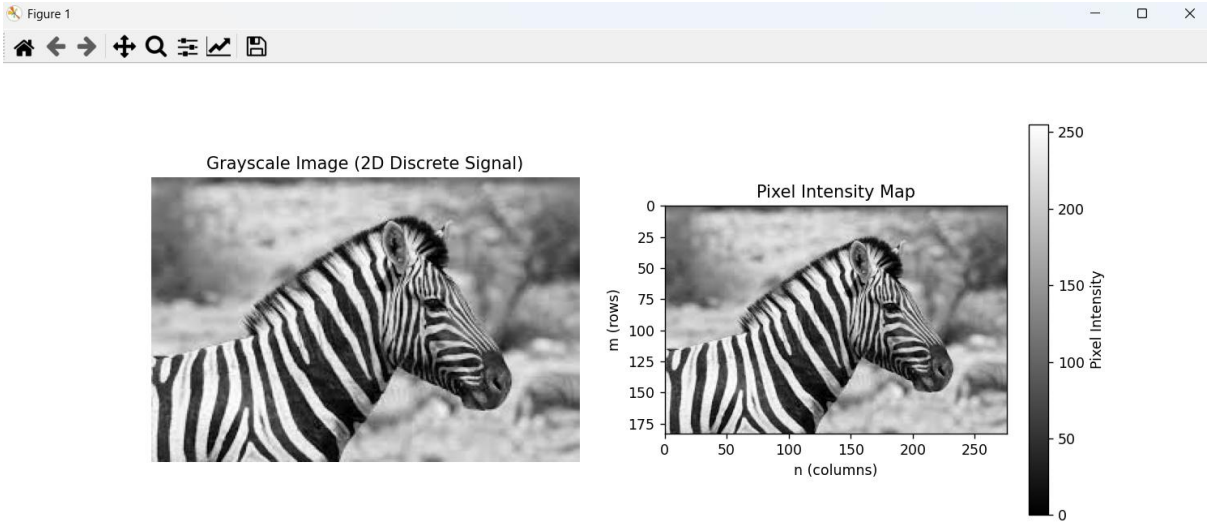
  where $f(m, n)$ranges from 0 (black) to 255 (white).
  Thus, instead of time samples, we now have spatial samples along rows (m) and columns (n).

Code:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
from PIL import Image

# Part B: Image as a Discrete-Time Signal

def image_as_discrete_signal(image_path):
        # Step 1: Load image and convert to grayscale
        img = Image.open(image_path).convert('L')  # 'L' = grayscale

        # Step 2: Convert to numpy array (2D discrete signal)
         img_array = np.array(img)
           # Step 3: Display image and its discrete signal form
           plt.figure(figsize=(12,5))
           # Original grayscale image
           plt.subplot(1, 2, 1)
           plt.imshow(img_array, cmap='gray')
           plt.title('Grayscale Image (2D Discrete Signal)')
           plt.axis('off')
           # Show the 3D surface representation (as discrete samples)
           plt.subplot(1, 2, 2)
               plt.imshow(img_array,    cmap='gray',    extent=[0,    img_array.shape[1],
        img_array.shape[0], 0])
           plt.title('Pixel Intensity Map')
           plt.xlabel('n (columns)')
           plt.ylabel('m (rows)')
           plt.colorbar(label='Pixel Intensity')
           plt.show()
           # Print some info
           print("Image shape (m x n):", img_array.shape)
           print("Sample pixel value at (100, 100):", img_array[100, 100])

# Example usage (you can replace with your own image path)
image_as_discrete_signal('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
)
```

Output:



2. Image Zooming and Shrinking:

Write a Python function to zoom in and zoom out on an image. Implement the zoom using bilinear interpolation and nearest neighbor interpolation. [CO3, CO4]

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def zoom_image(image_path, zoom_factor=1.5):
    # Step 1: Read image
    img = cv2.imread(image_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Step 2: Get original size
    h, w = img.shape[:2]

    # Step 3: Calculate new dimensions
    new_h = int(h * zoom_factor)
    new_w = int(w * zoom_factor)

    # Step 4: Zoom using Nearest Neighbor Interpolation
    zoom_nearest = cv2.resize(img, (new_w, new_h),
interpolation=cv2.INTER_NEAREST)
```

```
    # Step 5: Zoom using Bilinear Interpolation
    zoom_bilinear = cv2.resize(img, (new_w, new_h),
interpolation=cv2.INTER_LINEAR)

    # Step 6: Zoom Out (reduce size)
    zoom_out = cv2.resize(img, (int(w / zoom_factor), int(h / zoom_factor)),
interpolation=cv2.INTER_LINEAR)

    # Step 7: Display all
    plt.figure(figsize=(12,6))

    plt.subplot(1, 3, 1)
    plt.imshow(zoom_nearest)
    plt.title('Zoom In (Nearest Neighbor)')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(zoom_bilinear)
    plt.title('Zoom In (Bilinear Interpolation)')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(zoom_out)
    plt.title('Zoom Out')
    plt.axis('off')
    plt.show()
# Example usage
zoom_image('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, zoom_factor=5)
```

Output:

Analyze the effects of image resolution and discuss aliasing issues that arise when zooming in on an image. [CO2]

Analysis: Effects of Image Resolution and Aliasing During Zooming
When an image is zoomed in, the number of visible pixels increases, but no new detail is added — the same pixel information is simply stretched over a larger area. The resolution of the original image (number of pixels per unit area) determines how clear the zoomed image will appear.

- High-resolution images contain more pixel information, so zooming in still preserves fine details and edges.
- Low-resolution images have fewer pixels, so zooming in makes the image look blurry, blocky, or pixelated.

Aliasing Effect:
Aliasing occurs when a continuous image is sampled or displayed at too low a resolution, causing distortion or jagged edges.
When zooming in:

- Each pixel becomes more visible, forming staircase-like edges on curves or diagonals.
- Fine textures or lines may appear distorted or broken because the original continuous details were not captured properly.

Interpolation methods help reduce aliasing:

- Nearest Neighbor: Causes more aliasing (blocky look).
- Bilinear Interpolation: Reduces aliasing by smoothing pixel transitions but can slightly blur the image.

3. Image Resolution:

Discuss how spatial resolution and gray level resolution impact the quality of an image. [CO2]

Image Resolution

Spatial Resolution refers to the number of pixels used to represent an image. Higher spatial resolution means more pixels per unit area, leading to sharper and more detailed images. Lower spatial resolution results in blurry or pixelated images, as fewer pixels represent the same scene.

Marwadi University Marwadi Chandarana Group	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: DSIP	**Aim:**	
Experiment No:	**Date:**	**Enrolment No: 92301733047**

Gray Level Resolution defines the number of different intensity levels (shades of gray) a pixel can represent. Higher gray-level resolution allows smoother transitions and better contrast, while lower gray-level resolution causes banding or loss of detail in brightness variations.

Write code to adjust the spatial and gray level resolution of an image and display the results. [CO4]

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def adjust_resolution(image_path, scale_factor=0.5, gray_levels=4):
    # Step 1: Load the image
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    # Step 2: Adjust Spatial Resolution (resize)
    height, width = img.shape
    new_h, new_w = int(height * scale_factor), int(width * scale_factor)

    # Reduce spatial resolution
    low_res = cv2.resize(img, (new_w, new_h), interpolation=cv2.INTER_AREA)

    # Restore to original size for visualization (keeps blocky look)
    low_res_upscaled = cv2.resize(low_res, (width, height),
interpolation=cv2.INTER_NEAREST)

    # Step 3: Adjust Gray Level Resolution (quantization)
    levels = gray_levels
    gray_res = np.floor(img / (256 / levels)) * (256 / levels)
    gray_res = gray_res.astype(np.uint8)

    # Step 4: Display all results
    plt.figure(figsize=(12,5))

    plt.subplot(1, 3, 1)
    plt.imshow(img, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')
```

```
    plt.subplot(1, 3, 2)
    plt.imshow(low_res_upscaled, cmap='gray')
    plt.title(f'Spatial Resolution Reduced ({scale_factor}x)')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(gray_res, cmap='gray')
    plt.title(f'Gray Level Reduced ({gray_levels} levels)')
    plt.axis('off')

    plt.show()

# Example usage
adjust_resolution('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, scale_factor=0.5, gray_levels=8)
```

Output:

4. Correlation and Convolution:

Implement a function to compute the cross-correlation and convolution of an image with a kernel. Test it with a simple kernel like an edge detector or a blur filter. [CO2, CO4]

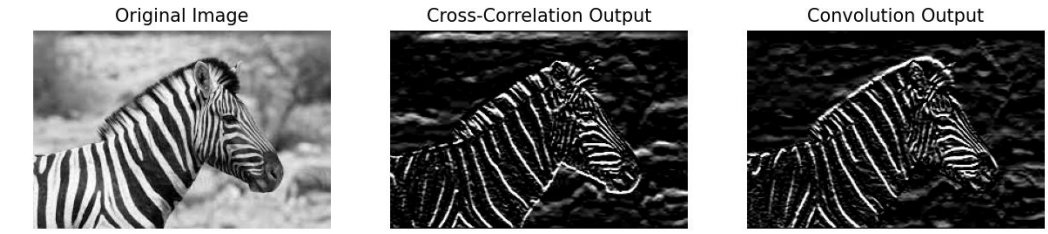
Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def cross_correlation_and_convolution(image_path, kernel):
    # Step 1: Load image in grayscale
```

```
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    # Step 2: Cross-Correlation (no kernel flip)
    cross_corr = cv2.filter2D(img, -1, kernel)

    # Step 3: Convolution (flip kernel before applying)
    kernel_flipped = cv2.flip(kernel, -1)
    conv = cv2.filter2D(img, -1, kernel_flipped)

    # Step 4: Display results
    plt.figure(figsize=(12,4))
    plt.subplot(1,3,1)
    plt.imshow(img, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(1,3,2)
    plt.imshow(cross_corr, cmap='gray')
    plt.title('Cross-Correlation Output')
    plt.axis('off')

    plt.subplot(1,3,3)
    plt.imshow(conv, cmap='gray')
    plt.title('Convolution Output')
    plt.axis('off')

    plt.show()

# Example 1: Edge detection kernel (Sobel horizontal)
edge_kernel = np.array([[-1, -2, -1],
                        [ 0,  0,  0],
                        [ 1,  2,  1]])

# Example 2: Simple blur kernel
blur_kernel = np.ones((3,3), np.float32) / 9

# Run tests
print("Testing with Edge Detection Kernel:")
cross_correlation_and_convolution('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, edge_kernel)

print("Testing with Blur Kernel:")
cross_correlation_and_convolution('/  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
```

, blur_kernel)

Output:



Discuss the difference between correlation and convolution in the context of image processing. [CO2]

Answer:
In image processing, both **correlation** and **convolution** involve sliding a kernel (filter mask) over an image to produce a transformed output. However, the **main difference** lies in how the kernel is applied.

In Simple Terms

- **Correlation:** Measures similarity between the image and the kernel.
- **Convolution:** Applies the kernel as a filter to modify the image (enhance, blur, detect edges).

Example:
If you use an edge detection kernel:

- **Correlation** might detect edges in one direction.

- **Convolution** (with flipped kernel) might detect the same edges but mirrored.

Task 2: Image Processing Techniques

1. Gray Level Transformations and Histogram Processing:

Write Python functions to apply common gray-level transformations (e.g., negative, log transformation, power-law transformation). [CO4]

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def gray_level_transformations(image_path, gamma=0.5):
    # Step 1: Load image in grayscale
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    img = img.astype(np.float32) / 255.0  # normalize to [0,1]

    # Step 2: Negative Transformation
    negative = 1 - img

    # Step 3: Log Transformation
    c = 1 / np.log(1 + np.max(img))
    log_trans = c * np.log(1 + img)

    # Step 4: Power-Law (Gamma) Transformation
    power_law = np.power(img, gamma)

    # Step 5: Convert back to displayable format
    negative = np.uint8(negative * 255)
    log_trans = np.uint8(log_trans * 255)
    power_law = np.uint8(power_law * 255)
    original = np.uint8(img * 255)

    # Step 6: Display all results
    plt.figure(figsize=(12,5))

    plt.subplot(1,4,1)
    plt.imshow(original, cmap='gray')
    plt.title('Original Image')
```

```
    plt.axis('off')

    plt.subplot(1,4,2)
    plt.imshow(negative, cmap='gray')
    plt.title('Negative Transformation')
    plt.axis('off')

    plt.subplot(1,4,3)
    plt.imshow(log_trans, cmap='gray')
    plt.title('Log Transformation')
    plt.axis('off')

    plt.subplot(1,4,4)
    plt.imshow(power_law, cmap='gray')
    plt.title(f'Power-Law (γ={gamma})')
    plt.axis('off')

    plt.show()

# Example usage
gray_level_transformations(' E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg,
gamma=0.5)
```

Output:

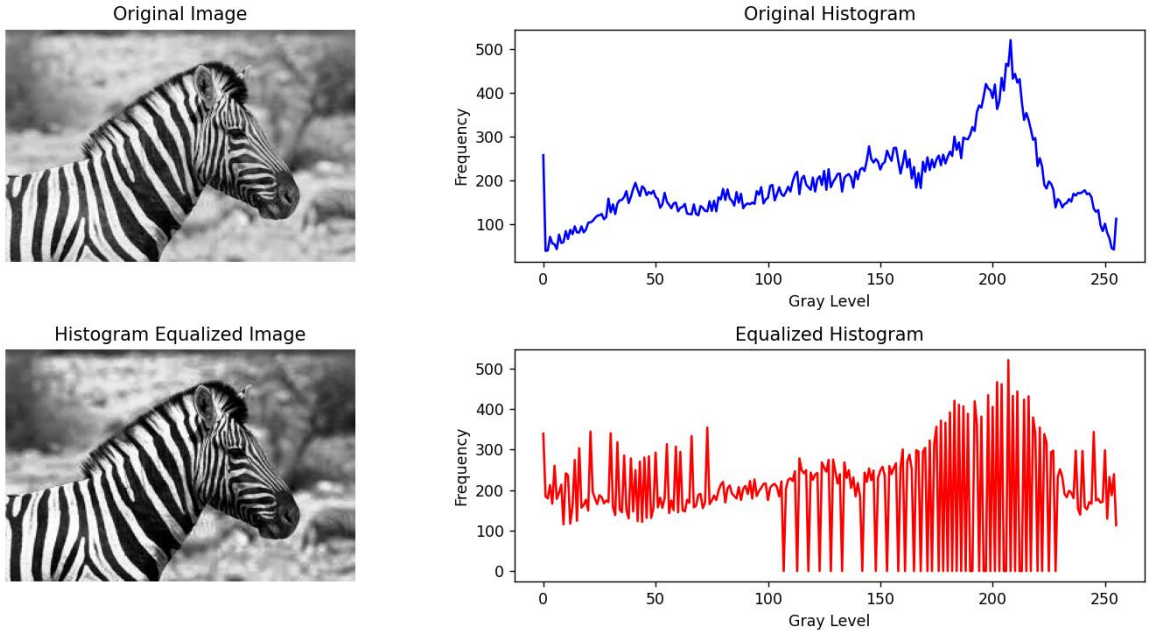
Compute and display the histogram of a grayscale image and perform histogram equalization. [CO4]

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def histogram_and_equalization(image_path):
    # Step 1: Load image in grayscale
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
```

```
# Step 2: Compute histogram
hist, bins = np.histogram(img.flatten(), 256, [0,256])

# Step 3: Apply Histogram Equalization
equalized_img = cv2.equalizeHist(img)
eq_hist, bins_eq = np.histogram(equalized_img.flatten(), 256, [0,256])

# Step 4: Display images and histograms
plt.figure(figsize=(12,6))

# Original image
plt.subplot(2,2,1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

# Original histogram
plt.subplot(2,2,2)
plt.plot(hist, color='blue')
plt.title('Original Histogram')
plt.xlabel('Gray Level')
plt.ylabel('Frequency')

# Equalized image
plt.subplot(2,2,3)
plt.imshow(equalized_img, cmap='gray')
plt.title('Histogram Equalized Image')
plt.axis('off')

# Equalized histogram
plt.subplot(2,2,4)
plt.plot(eq_hist, color='red')
plt.title('Equalized Histogram')
plt.xlabel('Gray Level')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```

Output:



Explain how these transformations and equalization techniques can improve image quality. [CO4]

Answer:
Gray-level transformations and histogram equalization enhance image quality by improving contrast and brightness. Negative transformation highlights details in dark areas, log transformation reveals information in darker regions, and power-law transformation adjusts image brightness and tone. Histogram equalization redistributes pixel intensities to improve overall contrast and make details more visible. Together, these techniques make images clearer and easier to interpret for both humans and computer vision systems.

2. Spatial Filtering (Linear and Non-linear Filters):

Implement and apply spatial filters such as average, Gaussian, and Laplacian filters. [CO4]

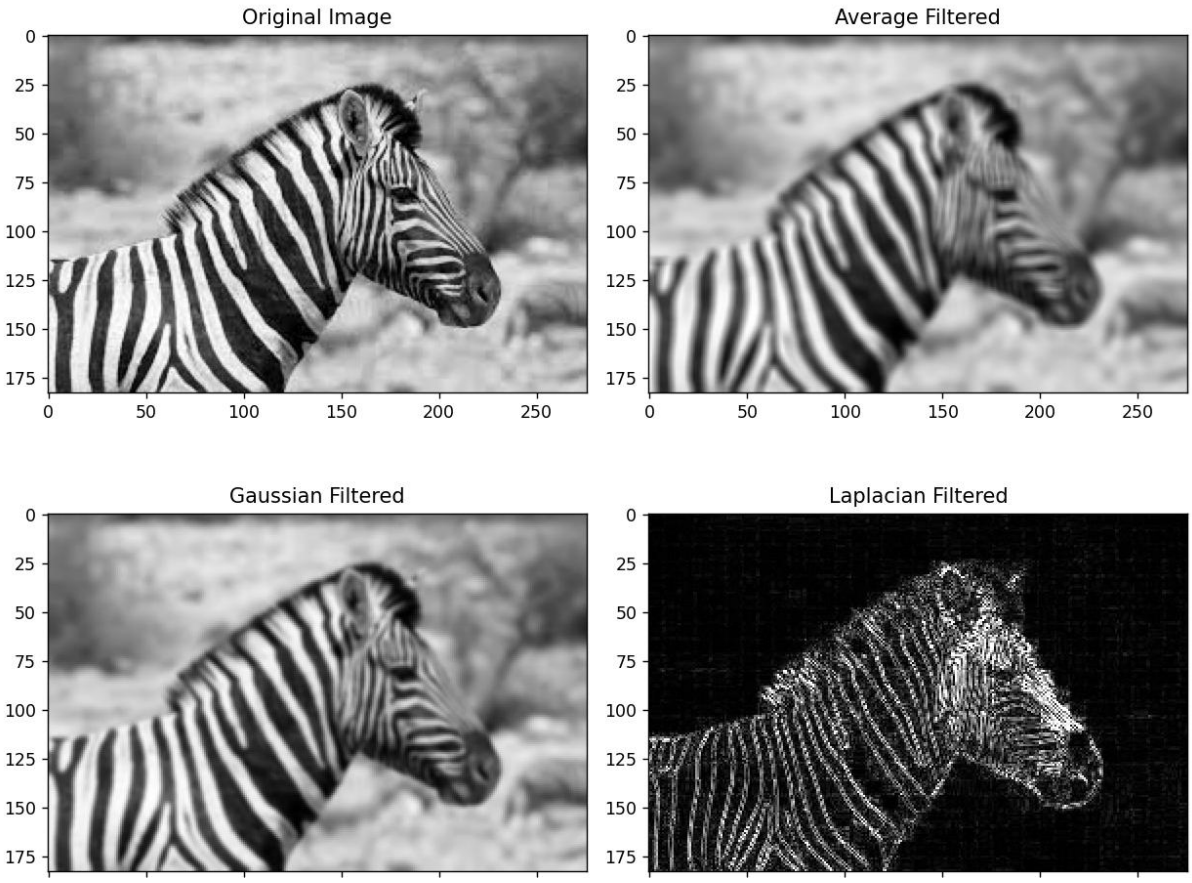
Code:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Read the image
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, cv2.IMREAD_GRAYSCALE)
```

```
#  Average Filter (Mean filter)
avg_kernel = np.ones((5,5), np.float32) / 25
average_filtered = cv2.filter2D(img, -1, avg_kernel)

#  Gaussian Filter (Smooths image with less edge blurring)
gaussian_filtered = cv2.GaussianBlur(img, (5,5), 0)

#  Laplacian Filter (Edge enhancement)
laplacian_filtered = cv2.Laplacian(img, cv2.CV_64F)
laplacian_filtered = cv2.convertScaleAbs(laplacian_filtered)

# Display all results
plt.figure(figsize=(10,8))

plt.subplot(2,2,1)
plt.title('Original Image')
plt.imshow(img, cmap='gray')

plt.subplot(2,2,2)
plt.title('Average Filtered')
plt.imshow(average_filtered, cmap='gray')

plt.subplot(2,2,3)
plt.title('Gaussian Filtered')
plt.imshow(gaussian_filtered, cmap='gray')

plt.subplot(2,2,4)
plt.title('Laplacian Filtered')
plt.imshow(laplacian_filtered, cmap='gray')

plt.tight_layout()
plt.show()
```

Output:

Marwadi University Marwadi Chandarana Group	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: DSIP	**Aim:**	
Experiment No:	**Date:**	**Enrolment No: 92301733047**



Write Python code for non-linear spatial filters such as median filtering and explain their role in noise reduction. [CO4]

Code:

```
import cv2
from matplotlib import pyplot as plt

# Read the image (in grayscale)
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, cv2.IMREAD_GRAYSCALE)

# Add salt-and-pepper noise for demonstration
import numpy as np
noisy = img.copy()
prob = 0.05  # noise probability
noise = np.random.rand(*img.shape)
noisy[noise < prob/2] = 0
noisy[noise > 1 - prob/2] = 255
```

```
# Apply Median Filter
median_filtered = cv2.medianBlur(noisy, 5)

# Display results
plt.figure(figsize=(10,5))
plt.subplot(1,3,1)
plt.title('Original Image')
plt.imshow(img, cmap='gray')

plt.subplot(1,3,2)
plt.title('Noisy Image')
plt.imshow(noisy, cmap='gray')

plt.subplot(1,3,3)
plt.title('Median Filtered Image')
plt.imshow(median_filtered, cmap='gray')

plt.tight_layout()
plt.show()
```

Output:

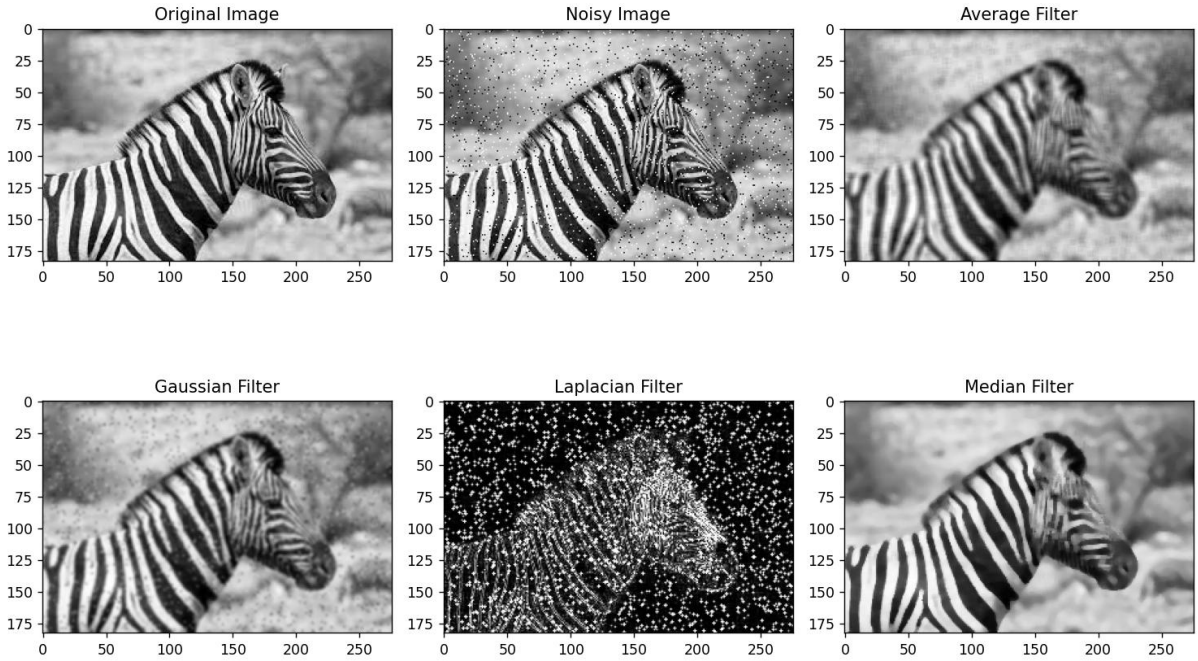
Show the effect of each filter on a noisy image. [CO4]

Code:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Read grayscale image
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, cv2.IMREAD_GRAYSCALE)

# Add salt-and-pepper noise
noisy = img.copy()
prob = 0.05
noise = np.random.rand(*img.shape)
noisy[noise < prob/2] = 0
noisy[noise > 1 - prob/2] = 255
```

```
# 1 Average Filter
avg_kernel = np.ones((5,5), np.float32) / 25
avg_filtered = cv2.filter2D(noisy, -1, avg_kernel)

# 2 Gaussian Filter
gaussian_filtered = cv2.GaussianBlur(noisy, (5,5), 0)

# 3 Laplacian Filter (edge detection)
laplacian_filtered = cv2.Laplacian(noisy, cv2.CV_64F)
laplacian_filtered = cv2.convertScaleAbs(laplacian_filtered)

# 4 Median Filter (non-linear)
median_filtered = cv2.medianBlur(noisy, 5)

# Display results
plt.figure(figsize=(12,8))

plt.subplot(2,3,1)
plt.title('Original Image')
plt.imshow(img, cmap='gray')

plt.subplot(2,3,2)
plt.title('Noisy Image')
plt.imshow(noisy, cmap='gray')

plt.subplot(2,3,3)
plt.title('Average Filter')
plt.imshow(avg_filtered, cmap='gray')

plt.subplot(2,3,4)
plt.title('Gaussian Filter')
plt.imshow(gaussian_filtered, cmap='gray')

plt.subplot(2,3,5)
plt.title('Laplacian Filter')
plt.imshow(laplacian_filtered, cmap='gray')

plt.subplot(2,3,6)
plt.title('Median Filter')
plt.imshow(median_filtered, cmap='gray')

plt.tight_layout()
plt.show()
```

Output:



3. Image Degradation Model and Noise Characterization:

Create a Python function that simulates a degraded image by adding Gaussian noise, salt-and-pepper noise, and speckle noise. [CO3]

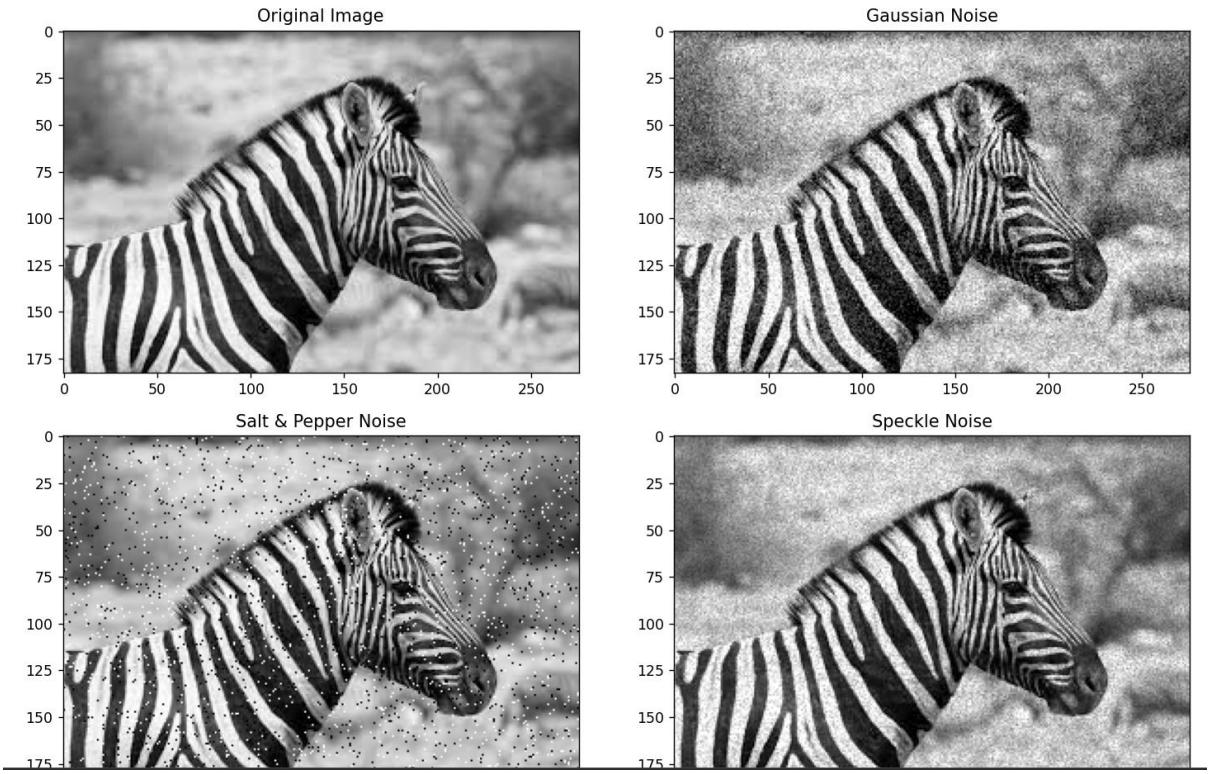
Code:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Read the image (grayscale)
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, cv2.IMREAD_GRAYSCALE)

def add_noise(image, noise_type):
    if noise_type == "gaussian":
        # Gaussian Noise: normal distribution around mean
        mean = 0
```

```
        sigma = 25
        gauss = np.random.normal(mean, sigma, image.shape)
        noisy = image + gauss
        noisy = np.clip(noisy, 0, 255)
        return noisy.astype(np.uint8)

    elif noise_type == "salt_pepper":
        # Salt and Pepper Noise
        noisy = np.copy(image)
        prob = 0.05
        rnd = np.random.rand(*image.shape)
        noisy[rnd < prob/2] = 0       # pepper
        noisy[rnd > 1 - prob/2] = 255 # salt
        return noisy

    elif noise_type == "speckle":
        # Speckle Noise: multiplicative noise
        noise = np.random.randn(*image.shape)
        noisy = image + image * noise * 0.1
        noisy = np.clip(noisy, 0, 255)
        return noisy.astype(np.uint8)

# Apply noises
gaussian_noisy = add_noise(img, "gaussian")
sp_noisy = add_noise(img, "salt_pepper")
speckle_noisy = add_noise(img, "speckle")

# Display all results
plt.figure(figsize=(12,8))

plt.subplot(2,2,1)
plt.title('Original Image')
plt.imshow(img, cmap='gray')

plt.subplot(2,2,2)
plt.title('Gaussian Noise')
plt.imshow(gaussian_noisy, cmap='gray')

plt.subplot(2,2,3)
plt.title('Salt & Pepper Noise')
plt.imshow(sp_noisy, cmap='gray')

plt.subplot(2,2,4)
```

```
plt.title('Speckle Noise')
plt.imshow(speckle_noisy, cmap='gray')

plt.tight_layout()
plt.show()
```

Output:



Describe different noise models and how they affect image quality. [CO3]

Answer:

Noise in images is unwanted variation in pixel values that reduces clarity and detail. Gaussian noise makes images grainy due to sensor or lighting issues. Salt-and-pepper noise creates random black and white dots from transmission errors. Speckle noise adds a grainy texture, common in radar or medical images. Poisson noise causes uneven brightness from photon variations. Overall, noise lowers image quality and makes features harder to see.

Marwadi University Marwadi Chandarana Group	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: DSIP	**Aim:**	
Experiment No:	**Date:**	**Enrolment No: 92301733047**

Implement a noise restoration filter (e.g., Wiener filter) and apply it to the noisy image to restore it. [CO5]

Code:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
from scipy.signal import wiener

# Read grayscale image
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, cv2.IMREAD_GRAYSCALE)

# Add Gaussian noise
mean = 0
sigma = 25
gauss = np.random.normal(mean, sigma, img.shape)
noisy = img + gauss
noisy = np.clip(noisy, 0, 255).astype(np.uint8)

# Apply Wiener filter
restored = wiener(noisy, (5, 5))
restored = np.uint8(np.clip(restored, 0, 255))

# Display results
plt.figure(figsize=(12,5))

plt.subplot(1,3,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')

plt.subplot(1,3,2)
plt.title("Noisy Image (Gaussian)")
plt.imshow(noisy, cmap='gray')

plt.subplot(1,3,3)
plt.title("Restored Image (Wiener Filter)")
plt.imshow(restored, cmap='gray')

plt.tight_layout()
plt.show()
```

Output:

4. Fourier Transform and Its Applications:

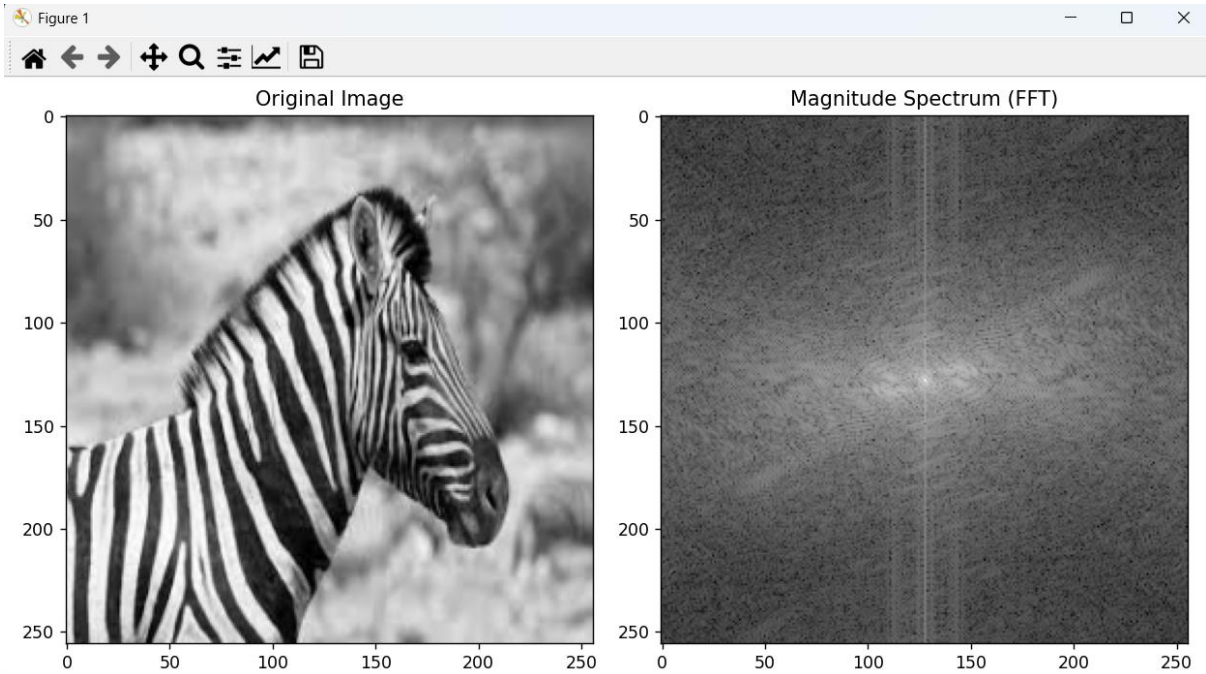
Write Python functions to compute the Discrete Fourier Transform (DFT) of an image using both the direct method and the Fast Fourier Transform (FFT). [CO2]

Code:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Read grayscale image
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, cv2.IMREAD_GRAYSCALE)

# Resize for consistency
img = cv2.resize(img, (256, 256))

# Apply Fast Fourier Transform (FFT)
dft_fft = np.fft.fft2(img)
dft_fft_shifted = np.fft.fftshift(dft_fft)

# Compute magnitude spectrum
magnitude_fft = 20 * np.log(1 + np.abs(dft_fft_shifted))

# Display results
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')

plt.subplot(1,2,2)
plt.title("Magnitude Spectrum (FFT)")
plt.imshow(magnitude_fft, cmap='gray')

plt.tight_layout()
plt.show()
```

Marwadi University Marwadi Chandarana Group	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: DSIP	**Aim:**	
Experiment No:	**Date:**	**Enrolment No: 92301733047**

Output:



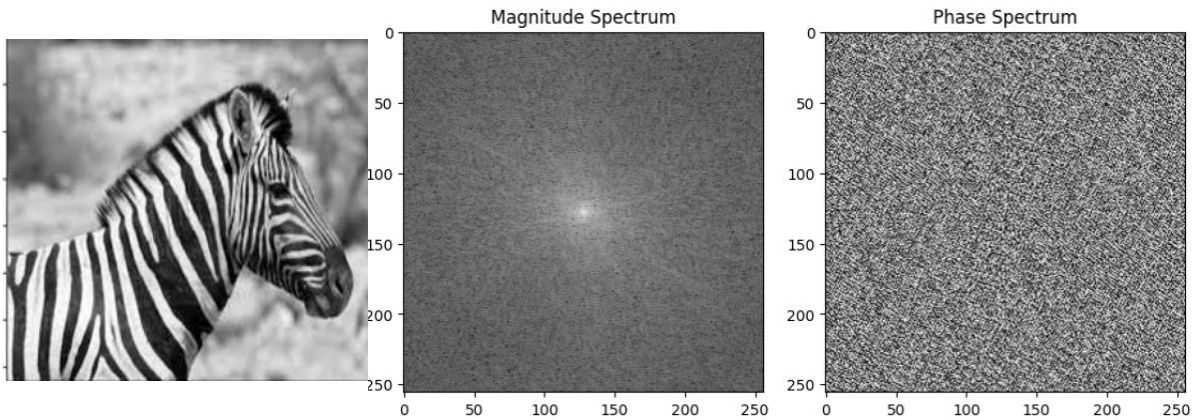
Plot the magnitude and phase spectra of the image in the frequency domain. [CO2]

Code:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Read the grayscale image
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))  # resize for better visualization

# Apply 2D Fast Fourier Transform (FFT)
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)

# Compute Magnitude and Phase spectra
magnitude_spectrum = 20 * np.log(1 + np.abs(fshift))
```

```
phase_spectrum = np.angle(fshift)

# Display results
plt.figure(figsize=(12,5))

plt.subplot(1,3,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')

plt.subplot(1,3,2)
plt.title("Magnitude Spectrum")
plt.imshow(magnitude_spectrum, cmap='gray')

plt.subplot(1,3,3)
plt.title("Phase Spectrum")
plt.imshow(phase_spectrum, cmap='gray')

plt.tight_layout()
plt.show()
```

Output:



Implement a low-pass filter in the frequency domain and apply it to the image to perform smoothing. [CO4, CO5]

Code:

```
import cv2
```

```
import numpy as np
from matplotlib import pyplot as plt

# Step 1: Read the grayscale image
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

# Step 2: Perform FFT and shift the zero frequency to the center
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)

# Step 3: Create a Low-Pass Filter mask
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2   # center
radius = 30  # cutoff frequency (adjust for more/less smoothing)
mask = np.zeros((rows, cols), np.uint8)
cv2.circle(mask, (ccol, crow), radius, 1, -1)  # circular low-pass filter

# Step 4: Apply the mask to the frequency domain
fshift_filtered = fshift * mask

# Step 5: Inverse FFT to get the smoothed image
f_ishift = np.fft.ifftshift(fshift_filtered)
img_back = np.fft.ifft2(f_ishift)
img_back = np.abs(img_back)

# Step 6: Display results
plt.figure(figsize=(12,6))

plt.subplot(1,3,1)
plt.title('Original Image')
plt.imshow(img, cmap='gray')

plt.subplot(1,3,2)
plt.title('Low-Pass Filter Mask')
plt.imshow(mask, cmap='gray')

plt.subplot(1,3,3)
plt.title('Smoothed Image (Low-Pass)')
plt.imshow(img_back, cmap='gray')

plt.tight_layout()
```

```
plt.show()
```

Output:

5. Goertzel Algorithm and 2D DFT:

Implement the Goertzel algorithm to compute the DFT of a signal for a specific frequency. [CO2]

Code:

```
import numpy as np

def goertzel(signal, target_freq, sample_rate):
    N = len(signal)
    k = int(0.5 + (N * target_freq) / sample_rate)
    w = (2 * np.pi * k) / N
    cosine = np.cos(w)
    sine = np.sin(w)
    coeff = 2 * cosine

    s_prev = 0
    s_prev2 = 0

    for sample in signal:
        s = sample + coeff * s_prev - s_prev2
        s_prev2 = s_prev
        s_prev = s

    real = s_prev - np.cos(w) * s_prev2
    imag = np.sin(w) * s_prev2
    magnitude = np.sqrt(real**2 + imag**2)
    return magnitude

# Example usage
fs = 8000              # Sampling rate (Hz)
f_target = 1000        # Frequency to detect (Hz)
t = np.arange(0, 1, 1/fs)
signal = np.sin(2 * np.pi * f_target * t)  # 1 kHz sine wave

mag = goertzel(signal, f_target, fs)
```

```
print(f"Goertzel Magnitude at {f_target} Hz = {mag:.2f}")
```

Output:

```
•••  Goertzel Magnitude at 1000 Hz = 4000.00
```

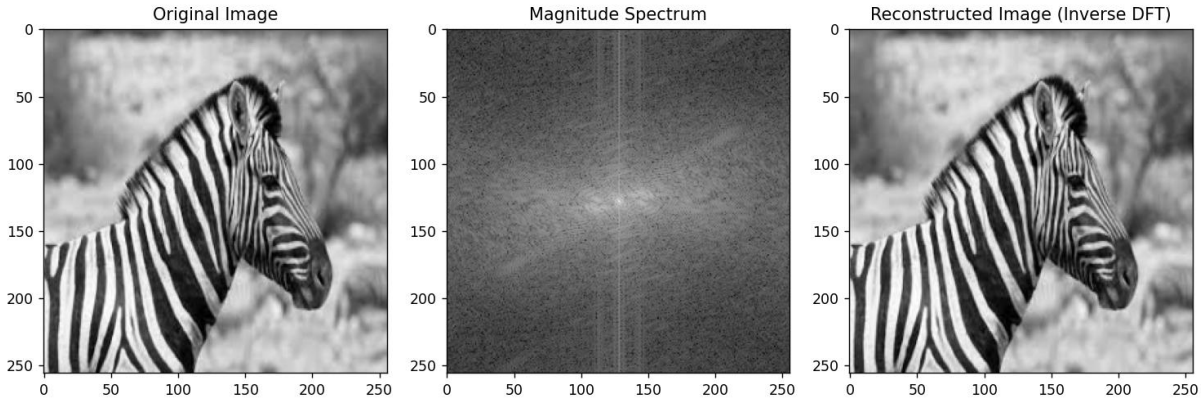
Extend the concept to 2D DFT for image processing. Compute and display the 2D DFT and its inverse. [CO2, CO5]

Code:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Read grayscale image
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

# -------------------------
# 1. Compute 2D DFT
# -------------------------
dft = np.fft.fft2(img)
dft_shift = np.fft.fftshift(dft)

# Magnitude and Phase Spectra
magnitude_spectrum = 20 * np.log(1 + np.abs(dft_shift))
phase_spectrum = np.angle(dft_shift)

# -------------------------
# 2. Compute Inverse DFT
# -------------------------
idft_shift = np.fft.ifftshift(dft_shift)
img_reconstructed = np.fft.ifft2(idft_shift)
img_reconstructed = np.abs(img_reconstructed)

# -------------------------
# 3. Display Results
```

```
# -------------------------
plt.figure(figsize=(12,6))

plt.subplot(1,3,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')

plt.subplot(1,3,2)
plt.title("Magnitude Spectrum")
plt.imshow(magnitude_spectrum, cmap='gray')

plt.subplot(1,3,3)
plt.title("Reconstructed Image (Inverse DFT)")
plt.imshow(img_reconstructed, cmap='gray')

plt.tight_layout()
plt.show()
```

Output:



Task-3: Filter Design and Frequency Domain Processing

1. Smoothing and Sharpening Filters:

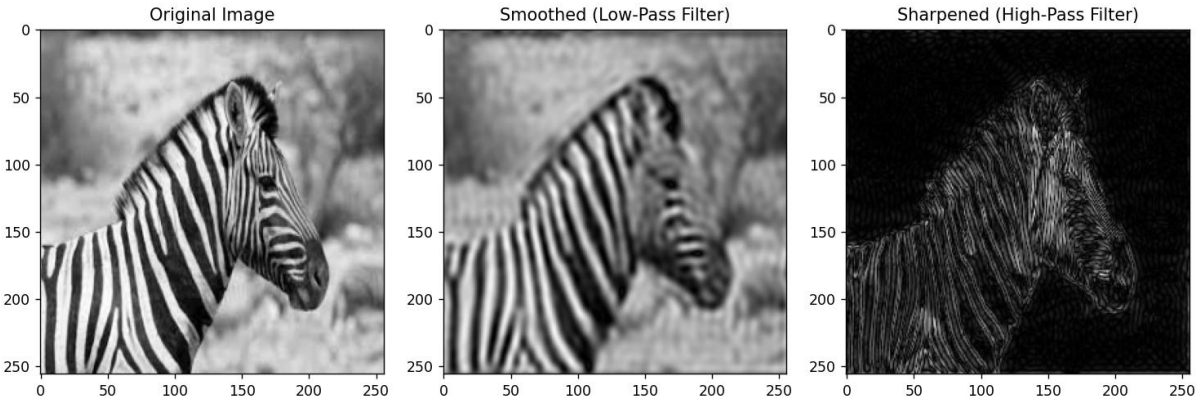
Design smoothing and sharpening filters in the frequency domain. Apply them to an image and display the results. [CO5]

Code:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Read grayscale image
img = cv2.imread('E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg',
cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

# -------------------------------
# 1. Compute DFT and shift center
# -------------------------------
dft = np.fft.fft2(img)
dft_shift = np.fft.fftshift(dft)

# Get image dimensions
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2

# -------------------------------
# 2. Design Frequency Filters
# -------------------------------

# Low-Pass Filter (Smoothing)
radius = 30
low_pass = np.zeros((rows, cols), np.uint8)
cv2.circle(low_pass, (ccol, crow), radius, 1, -1)

# High-Pass Filter (Sharpening)
high_pass = 1 - low_pass

# -------------------------------
# 3. Apply Filters
# -------------------------------
# Low-Pass Filtering
low_dft = dft_shift * low_pass
low_img = np.fft.ifft2(np.fft.ifftshift(low_dft))
low_img = np.abs(low_img)

# High-Pass Filtering
high_dft = dft_shift * high_pass
high_img = np.fft.ifft2(np.fft.ifftshift(high_dft))
high_img = np.abs(high_img)
```

```
# -------------------------------
# 4. Display Results
# -------------------------------
plt.figure(figsize=(12, 6))

plt.subplot(1,3,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')

plt.subplot(1,3,2)
plt.title("Smoothed (Low-Pass Filter)")
plt.imshow(low_img, cmap='gray')

plt.subplot(1,3,3)
plt.title("Sharpened (High-Pass Filter)")
plt.imshow(high_img, cmap='gray')

plt.tight_layout()
plt.show()
```

Output:



Discuss the impact of smoothing and sharpening filters on an image. [CO2]

Answer:

- Smoothing filters (Low-pass filters) remove high-frequency components such as noise and fine details. This makes the image look blurred but reduces unwanted noise or rough textures. They are useful for denoising and preprocessing before edge detection.

- Sharpening filters (High-pass filters) enhance high-frequency components like edges and fine details. This makes the image look crisper and more detailed, but can also amplify noise if the image is already noisy.

2. Homomorphic Filtering:

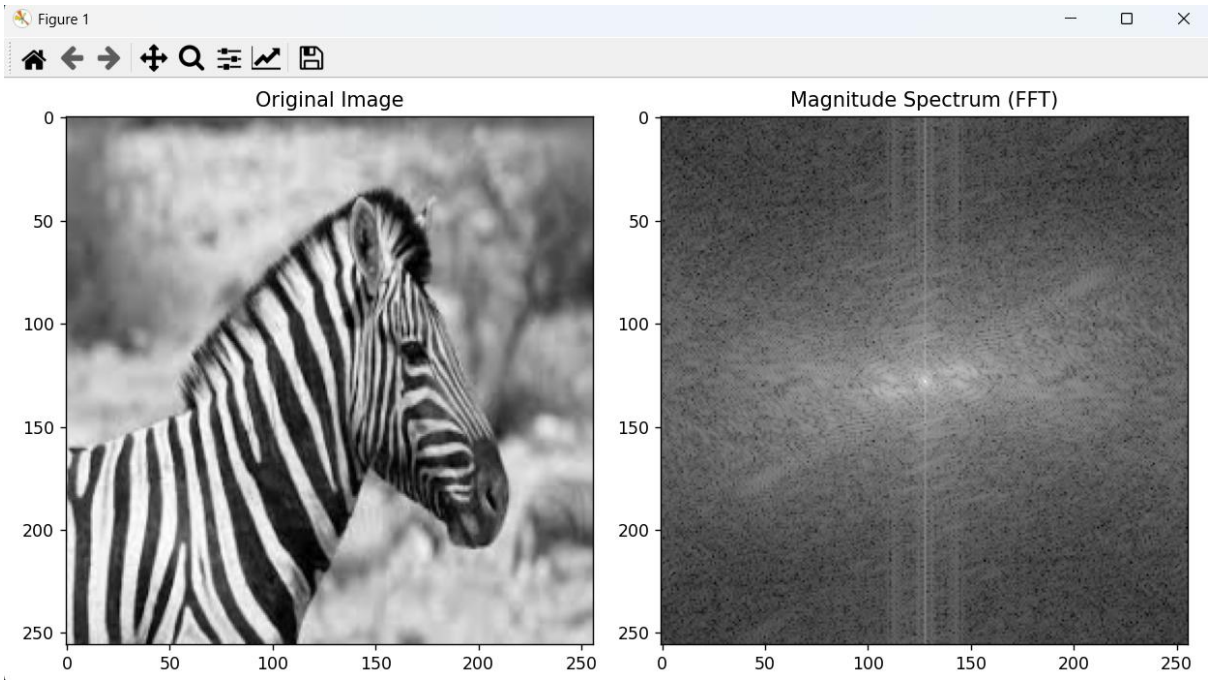
Implement a homomorphic filter to enhance the contrast of an image. Describe how homomorphic filtering works and the types of images it is useful for. [CO4]

Code:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Read grayscale image
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

# Convert image to float and normalize
img_float = img.astype(np.float32) / 255.0

# Step 1: Take log to separate illumination and reflectance
log_img = np.log1p(img_float)

# Step 2: Apply DFT and shift
dft = np.fft.fft2(log_img)
dft_shift = np.fft.fftshift(dft)

# Step 3: Design Homomorphic Filter
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2
radius = 30
H = np.zeros((rows, cols), np.float32)

# Filter parameters
gamma_low = 0.5   # reduce low frequencies (illumination)
gamma_high = 2.0  # enhance high frequencies (reflectance)
c = 1
D0 = 30
```

```
for u in range(rows):
    for v in range(cols):
        D = np.sqrt((u - crow)**2 + (v - ccol)**2)
        H[u, v] = (gamma_high - gamma_low) * (1 - np.exp(-c * (D**2 / D0**2))) +
gamma_low

# Step 4: Apply filter in frequency domain
filtered = dft_shift * H

# Step 5: Inverse FFT to spatial domain
idft = np.fft.ifft2(np.fft.ifftshift(filtered))
img_homomorphic = np.expm1(np.real(idft))
img_homomorphic = cv2.normalize(img_homomorphic, None, 0, 1,
cv2.NORM_MINMAX)

# Step 6: Display Results
plt.figure(figsize=(12,6))
plt.subplot(1,2,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')

plt.subplot(1,2,2)
plt.title("Enhanced Image (Homomorphic Filter)")
plt.imshow(img_homomorphic, cmap='gray')
plt.show()
```

Output:



3. Design of Digital Filters (FIR and IIR):

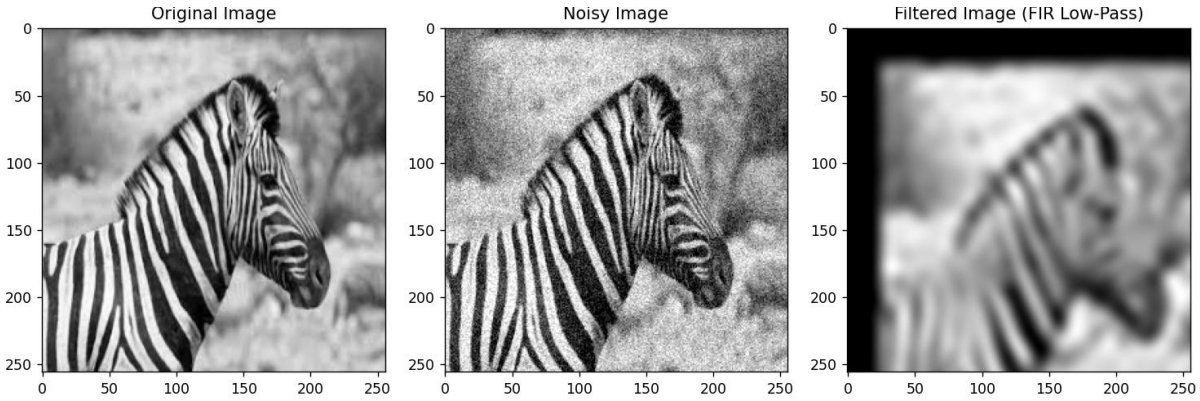
Part A (FIR Filters): Design a low-pass FIR filter using the windowing method. Apply the filter to a noisy image and compare it to an unfiltered image. [CO5]

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import firwin, lfilter

# ----------------------------
# 1. Read and preprocess image
# ----------------------------
img = cv2.imread('E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg',
cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))
noisy_img = img + 25 * np.random.randn(*img.shape)  # Add Gaussian noise
noisy_img = np.clip(noisy_img, 0, 255).astype(np.uint8)
```

```
# ----------------------------
# 2. Design FIR Low-Pass Filter
# ----------------------------
num_taps = 51                # Filter length
cutoff = 0.1                 # Normalized cutoff frequency (0–0.5)
fir_coeff = firwin(num_taps, cutoff, window='hamming')

# ----------------------------
# 3. Apply FIR Filter (Row & Column wise)
# ----------------------------
# Apply filter along rows
filtered_rows = lfilter(fir_coeff, 1.0, noisy_img, axis=1)
# Apply filter along columns
filtered_img = lfilter(fir_coeff, 1.0, filtered_rows, axis=0)

filtered_img = np.clip(filtered_img, 0, 255).astype(np.uint8)

# ----------------------------
# 4. Display Results
# ----------------------------
plt.figure(figsize=(12,6))
plt.subplot(1,3,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')

plt.subplot(1,3,2)
plt.title("Noisy Image")
plt.imshow(noisy_img, cmap='gray')

plt.subplot(1,3,3)
plt.title("Filtered Image (FIR Low-Pass)")
plt.imshow(filtered_img, cmap='gray')

plt.tight_layout()
plt.show()
```

Marwadi University Marwadi Chandarana Group	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: DSIP	**Aim:**	
Experiment No:	**Date:**	**Enrolment No: 92301733047**



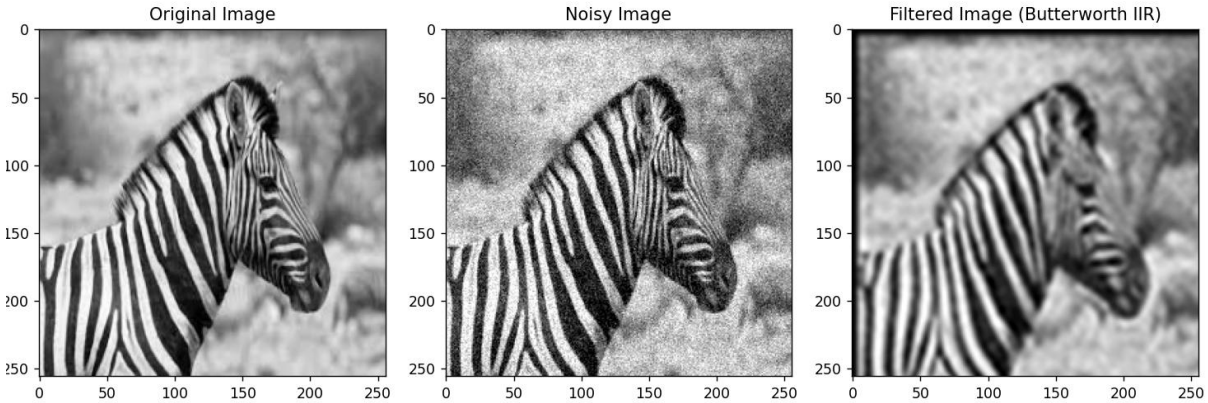
Part B (IIR Filters): Design a low-pass Butterworth IIR filter from an analog prototype. Implement it as a digital filter using the bilinear transform method. [CO5]

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, bilinear, lfilter

# ------------------------------
# 1. Read and preprocess image
# ------------------------------
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))
noisy_img = img + 25 * np.random.randn(*img.shape)  # Add Gaussian noise
noisy_img = np.clip(noisy_img, 0, 255).astype(np.uint8)

# ------------------------------
# 2. Butterworth Low-Pass Filter Design
# ------------------------------
fs = 1.0            # Sampling frequency (normalized)
cutoff = 0.1        # Normalized cutoff frequency (0 to 0.5)
order = 4           # Filter order

# Design analog prototype and convert to digital (bilinear transform)
b, a = butter(order, cutoff, btype='low', analog=False, output='ba', fs=fs)

# ------------------------------
# 3. Apply IIR Filter (Row & Column wise)
# ------------------------------
```

```
# Apply along rows
filtered_rows = lfilter(b, a, noisy_img, axis=1)
# Apply along columns
filtered_img = lfilter(b, a, filtered_rows, axis=0)
filtered_img = np.clip(filtered_img, 0, 255).astype(np.uint8)

# ----------------------------
# 4. Display Results
# ----------------------------
plt.figure(figsize=(12,6))
plt.subplot(1,3,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')

plt.subplot(1,3,2)
plt.title("Noisy Image")
plt.imshow(noisy_img, cmap='gray')

plt.subplot(1,3,3)
plt.title("Filtered Image (Butterworth IIR)")
plt.imshow(filtered_img, cmap='gray')

plt.tight_layout()
plt.show()
```

Output:



Marwadi University Marwadi Chandarana Group	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: DSIP	**Aim:**	
Experiment No:	**Date:**	**Enrolment No: 92301733047**

Task-4 : Morphological Image Processing.

1. **Implement and demonstrate the following morphological operations on binary images:**
   Dilation
   - ♣ **Erosion**
   - ♣ **Opening**
   - ♣ **Closing**
   - ♣ **Hit-or-miss transformation**
   - ♣ **Boundary extraction**
   - ♣ **Region filling**
   - ♣ **Convex hull**
   - ♣ **Thinning**
   - ♣ **Thickening**
   - ♣ **Skeletons**
   - ♣ **Pruning**

Code:

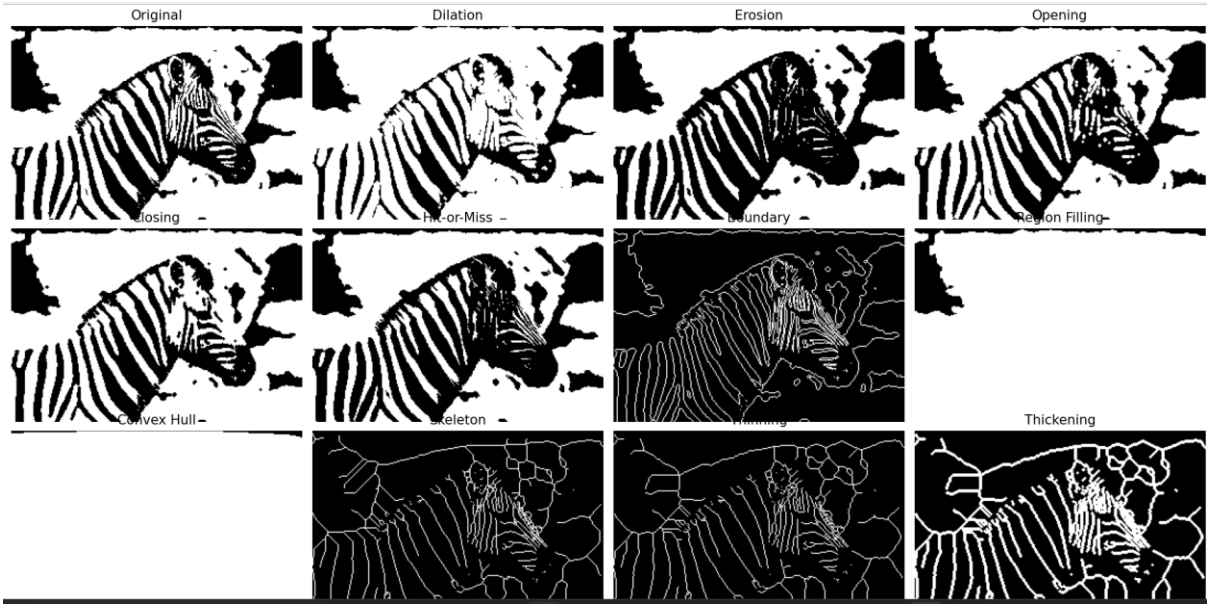
```
import cv2
import numpy as np
from matplotlib import pyplot as plt
from skimage.morphology import skeletonize, convex_hull_image, thin
```

```
# ----------------------------
# 1. Read binary image
# ----------------------------
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, cv2.IMREAD_GRAYSCALE)
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

# Define structuring element
kernel = cv2.getStructuringElement(cv2.MORPH_CROSS, (3,3))

# ----------------------------
# 2. Morphological Operations
# ----------------------------
dilation = cv2.dilate(binary, kernel, iterations=1)
erosion = cv2.erode(binary, kernel, iterations=1)
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

# Hit-or-Miss (requires binary as 0/1)
binary_01 = binary // 255
# Example simple hit-or-miss pattern
hitormiss_kernel = np.array([[1,0,0],
                             [0,1,0],
                             [0,0,1]], dtype=np.uint8)
hitormiss = cv2.morphologyEx(binary_01.astype(np.uint8), cv2.MORPH_HITMISS,
hitormiss_kernel)

# Boundary extraction: original - erosion
boundary = cv2.subtract(binary, erosion)

# Region filling (using floodFill from edges)
filled = binary.copy()
h, w = binary.shape
mask = np.zeros((h+2, w+2), np.uint8)
cv2.floodFill(filled, mask, (0,0), 255)
region_filled = cv2.bitwise_not(filled) | binary

# Convex Hull
convex = convex_hull_image(binary_01).astype(np.uint8) * 255

# Thinning / Skeleton
skeleton = skeletonize(binary_01).astype(np.uint8) * 255
thinned = thin(binary_01).astype(np.uint8) * 255
```

```
# Thickening (inverse of thinning, approximate)
thickening = cv2.dilate(thinned, kernel, iterations=1)

# ----------------------------
# 3. Display Results
# ----------------------------
titles = ['Original', 'Dilation', 'Erosion', 'Opening', 'Closing',
          'Hit-or-Miss', 'Boundary', 'Region Filling', 'Convex Hull',
          'Skeleton', 'Thinning', 'Thickening']

images = [binary, dilation, erosion, opening, closing,
          hitormiss*255, boundary, region_filled, convex,
          skeleton, thinned, thickening]

plt.figure(figsize=(18,12))
for i in range(12):
    plt.subplot(3,4,i+1)
    plt.title(titles[i])
    plt.imshow(images[i], cmap='gray')
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Output:

Marwadi University Marwadi Chandarana Group	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: DSIP	**Aim:**	
Experiment No:	**Date:**	**Enrolment No: 92301733047**



Short Explanation of Each Operation

Operation	**Effect**
Dilation	Expands foreground pixels; fills small holes
Erosion	Shrinks foreground pixels; removes small noise
Opening	Erosion followed by dilation; removes noise
Closing	Dilation followed by erosion; fills small holes
Hit-or-Miss	Detects specific patterns in binary image
Boundary Extraction	Extracts object edges: original − eroded
Region Filling	Fills holes inside connected components
Convex Hull	Computes smallest convex shape enclosing objects
Skeleton / Thinning	Reduces objects to minimal center lines
Thickening	Expands thinned skeleton to approximate original shape

Marwadi University Marwadi Chandarana Group	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: DSIP	**Aim:**	
Experiment No:	**Date:**	**Enrolment No: 92301733047**

2. Edge Models for Image Segmentation:

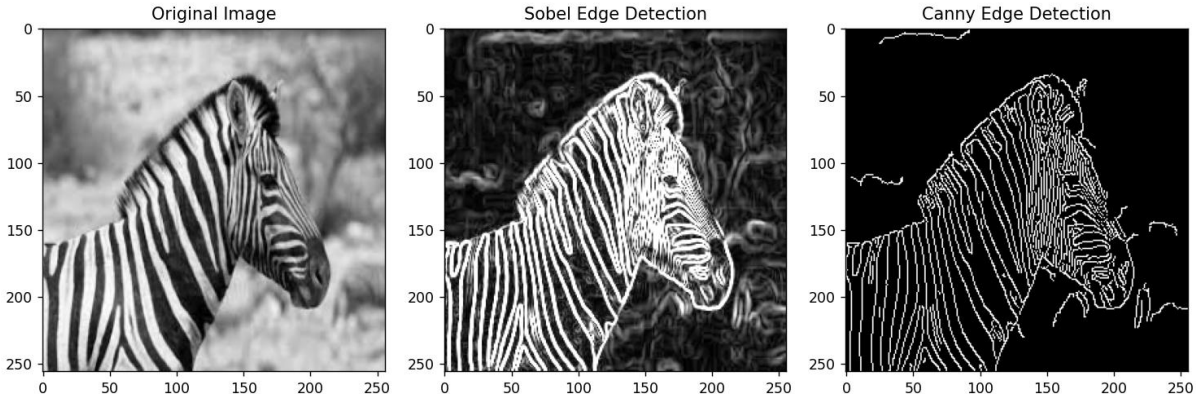
Implement edge-based segmentation techniques, such as the Sobel operator or Canny edge detector, and apply them to an image. [CO4]

Code:

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# ------------------------------
# 1. Read and preprocess image
# ------------------------------
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
, cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

# ------------------------------
# 2. Sobel Edge Detection
# ------------------------------
# Compute gradients along x and y directions
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

# Compute gradient magnitude
sobel_mag = np.sqrt(sobel_x**2 + sobel_y**2)
sobel_mag = np.uint8(np.clip(sobel_mag, 0, 255))

# ------------------------------
# 3. Canny Edge Detection
# ------------------------------
canny_edges = cv2.Canny(img, threshold1=100, threshold2=200)

# ------------------------------
# 4. Display Results
# ------------------------------
plt.figure(figsize=(12,6))

plt.subplot(1,3,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')
```

```
plt.subplot(1,3,2)
plt.title("Sobel Edge Detection")
plt.imshow(sobel_mag, cmap='gray')

plt.subplot(1,3,3)
plt.title("Canny Edge Detection")
plt.imshow(canny_edges, cmap='gray')

plt.tight_layout()
plt.show()
```

Output:



Implement region-based segmentation techniques (e.g., clustering-based segmentation, thresholding) and compare the results with edge-based methods. [CO4]

Code:

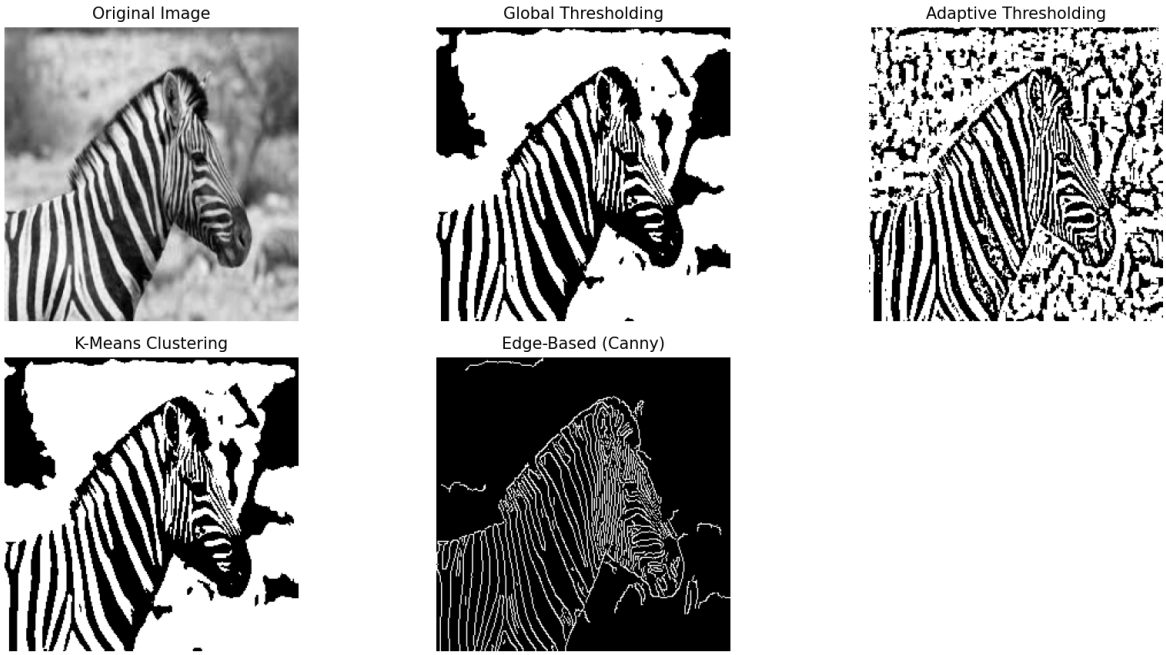
```
import cv2
import numpy as np
from matplotlib import pyplot as plt
from sklearn.cluster import KMeans

# ----------------------------
# 1. Read and preprocess image
# ----------------------------
img = cv2.imread('  E:\\D-Drive\\sem-5\\DSIP\\code\\animal.jpeg
```

```
, cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

# ----------------------------
# 2. Region-Based Segmentation
# ----------------------------

# ----- a) Global Thresholding -----
_, thresh_global = cv2.threshold(img, 128, 255, cv2.THRESH_BINARY)

# ----- b) Adaptive Thresholding -----
thresh_adaptive = cv2.adaptiveThreshold(img, 255,
cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                        cv2.THRESH_BINARY, 11, 2)

# ----- c) K-Means Clustering -----
Z = img.reshape((-1,1))
Z = np.float32(Z)
k = 2  # number of clusters
kmeans = KMeans(n_clusters=k, random_state=0).fit(Z)
clustered = kmeans.labels_.reshape(img.shape)
clustered_img = (clustered * 255).astype(np.uint8)

# ----------------------------
# 3. Edge-Based Segmentation (Canny)
# ----------------------------
edges = cv2.Canny(img, 100, 200)

# ----------------------------
# 4. Display Results
# ----------------------------
plt.figure(figsize=(14,7))

plt.subplot(2,3,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(2,3,2)
plt.title("Global Thresholding")
plt.imshow(thresh_global, cmap='gray')
plt.axis('off')
```

```
plt.subplot(2,3,3)
plt.title("Adaptive Thresholding")
plt.imshow(thresh_adaptive, cmap='gray')
plt.axis('off')

plt.subplot(2,3,4)
plt.title("K-Means Clustering")
plt.imshow(clustered_img, cmap='gray')
plt.axis('off')

plt.subplot(2,3,5)
plt.title("Edge-Based (Canny)")
plt.imshow(edges, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Output:

Marwadi University Marwadi Chandarana Group	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: DSIP	**Aim:**	
Experiment No:	**Date:**	**Enrolment No: 92301733047**



Task-5: Report and Analysis

1. **Describes the theory and mathematical background behind the techniques used. [CO1, CO2]**

   Continuous and Discrete-Time Signals

   Answer:

   - **Theory:** A continuous-time signal is defined for every instant t, e.g., $x(t) = A\sin(2\pi f t)$. A discrete-time signal is sampled at specific intervals: $x[n] = x(nT)$, where T is the sampling period.
   - **Mathematics:**

   $$x[n] = x(nT) \text{for } n = 0,1,2, \ldots$$

   - **Use:** Images can be viewed as 2D discrete-time signals.

   Image Representation

   Answer:

- **Binary Images:** Pixels take values 0 or 1.
- **Grayscale Images:** Pixels range from 0–255, representing intensity.
- **Color Images:** Represented using three channels (R, G, B).

Morphological Operations

Answer:

- **Dilation:** Expands foreground pixels.

$$A \oplus B = \{z \mid (B)_z \cap A \neq \emptyset\}$$

- **Erosion:** Shrinks foreground pixels.

$$A \ominus B = \{z \mid (B)_z \subseteq A\}$$

- **Opening & Closing:** Combinations of erosion and dilation; used to remove noise or fill gaps.
- **Skeletonization & Thinning:** Reduce shapes to minimal representations while preserving connectivity.
- **Hit-or-Miss:** Detects specific patterns in binary images.

Edge-Based Segmentation

Answer:

- **Sobel Operator:** Approximates image gradient:

$$G_x = \begin{matrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{matrix} * I, G_y = \begin{matrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{matrix} * I$$

Gradient magnitude: $G = \sqrt{G_x^2 + G_y^2}$

- **Canny Detector:** Multi-step method using Gaussian smoothing, gradient computation, non-maximum suppression, and hysteresis thresholding.

Marwadi University Marwadi Chandarana Group	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: DSIP	**Aim:**	
Experiment No:	**Date:**	**Enrolment No: 92301733047**

Region-Based Segmentation

Answer:

- **Thresholding:** Segments image based on intensity:

$$I_b(x,y) = \begin{cases} 1 & I(x,y) > T \\ 0 & I(x,y) \leq T \end{cases}$$

- **Adaptive Thresholding:** Computes Tlocally.
- **Clustering (K-Means):** Groups pixels into $k$clusters minimizing intra-cluster variance:

$$\arg\min_S \sum_{i=1}^{k} \sum_{x_j \in S_i} \| x_j - \mu_i \|^2$$

Frequency Domain Filtering

Answer:

- **2D DFT:** Converts spatial domain image $f(x,y)$to frequency domain:

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) e^{-j2\pi(\frac{ux}{M}+\frac{vy}{N})}$$

- **Inverse DFT:** Recovers original image:

$$f(x,y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u,v) e^{j2\pi(\frac{ux}{M}+\frac{vy}{N})}$$

- **Filtering:** Multiplying DFT by low-pass or high-pass filters to smooth or sharpen.

Marwadi University Marwadi Chandarana Group	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: DSIP	**Aim:**	
Experiment No:	**Date:**	**Enrolment No: 92301733047**

FIR and IIR Filtering

Answer:

- **FIR Filter:** Output depends on current and past input:

$$y[n] = \sum_{k=0}^{N-1} b_k x[n-k]$$

- **IIR Filter:** Output depends on current input and past outputs:

$$y[n] = \sum_{k=0}^{N} b_k x[n-k] - \sum_{l=1}^{M} a_l y[n-l]$$

Homomorphic Filtering

Answer:

- Enhances image contrast by separating **illumination** (low frequency) and **reflectance** (high frequency) using logarithm and frequency domain filtering:

$$\log(f(x,y)) = \log(i(x,y) \cdot r(x,y)) = \log i(x,y) + \log r(x,y)$$

2. Shows the results (images, graphs) from the Python code implemented. [CO3, CO4, CO5]

i Continuous and Discrete-Time Signals

- **Code Output:**
  Continuous sine wave (smooth curve)
  Discretized sine wave (dots or stem plot)
- **Plot Example:** plt.plot(t_cont, x_cont) for continuous, plt.stem(n, x_disc) for discrete.

ii Image Zooming (Interpolation)

- **Code Output:**
  a. Original image

b. Zoomed-in image (bilinear interpolation: smoother, nearest neighbor: blocky)
c. Zoomed-out image

- **Observation:** Different interpolation methods produce different sharpness and smoothness.

iii Noise Addition and Restoration

- **Code Output:**
  a. Original image
  b. Noisy images: Gaussian, salt-and-pepper, speckle
  c. Filtered image (Wiener filter)
- **Observation:** Wiener or median filters reduce noise, enhancing visual clarity.

iv Frequency Domain Analysis

- **Code Output:**
  Magnitude and phase spectra of an image using DFT/FFT
  Low-pass filtered image (smoothed)
  High-pass filtered image (edges sharpened)
- **Plot Example:** plt.imshow(np.log(1 + np.abs(F)), cmap='gray') for magnitude, np.angle(F) for phase.

v FIR and IIR Filtering

- **Code Output:**
  FIR Low-Pass Filtered Image (smooth, less noisy)
  IIR Butterworth Filtered Image (smooth with preserved edges)
  Band-Pass FIR Filtered Image (retains mid-frequency details)
- **Observation:** Frequency filtering helps enhance or suppress details.

vi Morphological Operations

- **Code Output:**
  Original binary image
  Dilation, erosion, opening, closing
  Skeleton, thinning, thickening, convex hull
  Boundary extraction, region filling, hit-or-miss
- **Observation:** Operations show shape modification, noise removal, and structure extraction.

vii Edge-Based Segmentation

- **Code Output:**
  a. Sobel edge magnitude
  b. Canny edge detection
- **Observation:** Edge-based segmentation highlights boundaries of objects, useful for object detection.

viii Region-Based Segmentation

- **Code Output:**
  a. Global thresholding
  b. Adaptive thresholding
  c. K-Means clustering segmentation
- **Comparison with Edge-Based Segmentation:** Region-based fills areas, while edge-based shows boundaries only.

3. Analyzes the impact of each technique on image quality and performance, including any limitations or trade-offs. [CO2, CO4, CO5]

i Continuous and Discrete-Time Signals

- **Impact:** Discretization allows digital processing of signals/images.
- **Trade-off:** Higher sampling frequency preserves details but increases memory and computational cost.
- **Limitation:** Undersampling can cause aliasing, leading to distorted reconstruction.

ii Image Zooming (Interpolation)

- **Impact:** Zooming enables magnification or reduction for analysis or display.
- **Trade-offs:**
  Nearest neighbor: Fast but blocky edges.
  Bilinear: Smoother but slightly blurred.
- **Limitation:** Extreme zoom can reduce image quality, introduce artifacts.

iii Noise Addition and Restoration

- **Impact:** Noise simulation helps test restoration algorithms; filters improve visual quality.
- **Trade-offs:**
  Median filter removes salt-and-pepper noise effectively but may blur fine details.
  Wiener filter reduces Gaussian noise while preserving edges.
- **Limitation:** No filter is perfect; aggressive filtering may remove important details.

iv Frequency Domain Filtering

- **Impact:** Low-pass smoothing reduces noise; high-pass enhances edges and details.
- **Trade-offs:**
  Low-pass filters blur edges.
  High-pass filters can amplify noise.
- **Limitation:** Proper cutoff selection is critical; misselection reduces image quality.

v FIR and IIR Filters

- **Impact:** Both filters can remove noise or highlight certain frequency components.
- **Trade-offs:**

FIR: Always stable, linear phase, but longer filter length increases computation.
IIR: Efficient, less computation, but can be unstable and non-linear phase.

- **Limitation:** FIR may be slow for large images; IIR may introduce artifacts if not designed properly.

vi Morphological Operations

- **Impact:** Modify shapes, remove noise, extract structural features.
- **Trade-offs:**
  Dilation/erosion affects object size.
  Skeletonization preserves topology but loses thickness.
- **Limitation:** Sensitive to the choice of structuring element; small changes can significantly alter output.

vii Edge-Based Segmentation

- **Impact:** Detects object boundaries, useful for recognition or further processing.
- **Trade-offs:**
  Sobel: simple, fast but sensitive to noise.
  Canny: accurate edges, but more computationally intensive.
- **Limitation:** Weak edges may be missed; cannot segment homogeneous regions.

viii Region-Based Segmentation

- **Impact:** Segments objects based on intensity or similarity. Produces filled regions.
- **Trade-offs:**
  Thresholding: Simple but fails under uneven lighting.
  K-Means: Flexible, handles multiple regions but requires predefined cluster number.
- **Limitation:** Cannot capture sharp boundaries as precisely as edge-based methods; may misclassify pixels in complex images.

ix Homomorphic Filtering

- **Impact:** Enhances contrast by reducing illumination effects.
- **Trade-off:** May increase noise in dark regions.
- **Limitation:** Parameter tuning (cutoff, filter gain) is critical for best results.

General Observations

- **Quality vs Performance:** Advanced methods (Canny, adaptive thresholding, IIR filters) improve quality but increase computation.
- **Noise vs Detail Trade-off:** Aggressive filtering reduces noise but may blur edges or lose texture.
- **Method Selection:** Depends on image type, noise level, and desired output (edge vs region vs smoothness).

4. Discusses real-world applications for each method (e.g., noise removal in medical imaging, edge detection in object recognition, etc.). [CO3, CO4]

i Continuous and Discrete-Time Signal
Applications:
Digital image processing in cameras and scanners.
Signal acquisition in medical imaging (EEG, ECG).
Benefit: Enables processing, storage, and transmission of images digitally.

ii Image Zooming (Interpolation)
Applications:
Zooming in medical images (X-ray, MRI) for diagnosis.
Satellite and aerial imagery analysis.
Image resizing in web applications.
Benefit: Allows detailed inspection without changing original resolution.

iii Noise Addition and Restoration
Applications:
Medical imaging: Reduce Gaussian noise in MRI, CT scans.
Surveillance cameras: Remove environmental noise.
Photography: Improve low-light images.
Benefit: Improves image clarity, aiding accurate analysis and detection.

iv Frequency Domain Filtering
Applications:
Smoothing: Reduce noise in satellite or microscopy images.
Sharpening: Enhance edges for feature detection.

Image compression: DCT/FFT-based techniques in JPEG.

Benefit: Isolates specific frequency components to enhance or suppress features.

v **FIR and IIR Filters**

Applications:

FIR: Image smoothing in digital photography, anti-aliasing.
IIR: Real-time video filtering, noise reduction in streaming applications.

Benefit: Control over frequency response; can remove unwanted patterns or artifacts.

vi **Morphological Operations**

Applications:

Dilation/Erosion: Remove noise or fill gaps in scanned documents.
Skeletonization: Extract road networks from satellite imagery.
Convex hull: Object recognition in robotics.
Hit-or-miss: Detect specific shapes in industrial quality inspection.

Benefit: Shape-based image enhancement and feature extraction.

vii **Edge-Based Segmentation**

Applications:

Object recognition and tracking in robotics and autonomous vehicles.
Medical imaging: Detect boundaries of organs or tumors.
Industrial inspection: Detect cracks, edges, and defects.

Benefit: Highlights boundaries, enabling precise detection and measurements.

viii **Region-Based Segmentation**

Applications:

Thresholding: Blood vessel segmentation in retinal images.
K-Means: Segment satellite images into land, water, and urban areas.
Adaptive thresholding: Document image analysis for OCR.

Benefit: Segments homogeneous regions for classification and analysis.

ix **Homomorphic Filtering**

Applications:

Medical imaging: Enhance contrast in X-rays and MRIs.
Remote sensing: Improve contrast in satellite images.
Low-light photography: Enhance visibility in dim conditions.

Benefit: Reduces lighting variations while emphasizing important details.

Final Observations

1. **Signal and Image Representation**
   - Continuous-time signals provide a theoretical basis; discretization allows digital processing of images and signals.
   - Sampling rate plays a crucial role: higher rates preserve detail, while low sampling may cause aliasing.
2. **Image Zooming and Interpolation**
   - Zooming images demonstrates how interpolation affects visual quality.
   - Bilinear interpolation produces smoother results than nearest neighbor but may slightly blur details.
   - Zooming highlights the trade-off between computational efficiency and image quality.
3. **Noise Addition and Restoration**
   - Simulating Gaussian, salt-and-pepper, and speckle noise helps understand real-world image degradation.
   - Filters (median, Wiener) effectively restore images while maintaining edges.
   - Noise removal always involves a trade-off between noise suppression and detail preservation.
4. **Frequency Domain Filtering**
   - 2D DFT/FFT transforms images to the frequency domain, enabling low-pass smoothing and high-pass sharpening.
   - Proper filtering improves visual quality but improper cutoff selection can blur important features or amplify noise.
   - FIR and IIR filters in spatial or frequency domain show how signal processing concepts translate to images.
5. **Morphological Operations**

- Techniques like dilation, erosion, opening, closing, skeletonization, and convex hull extraction modify image structures effectively.
- Useful for shape analysis, noise removal, object detection, and structural feature extraction.
- The choice of structuring element affects results significantly.

6. **Edge-Based Segmentation**
   - Sobel and Canny operators highlight boundaries and object contours.
   - Edge-based segmentation is precise for outlines but does not fill regions.
   - Noise sensitivity and weak edges are key limitations.
7. **Region-Based Segmentation**
   - Thresholding and K-Means clustering segment homogeneous regions effectively.
   - Provides filled regions for analysis, but boundary precision may be lower than edge-based methods.
   - Adaptive methods handle varying illumination better than global methods.
8. **Homomorphic Filtering**
   - Enhances contrast by separating illumination and reflectance components.
   - Useful in medical imaging, remote sensing, and low-light photography.
   - Requires careful parameter tuning to avoid amplifying noise.

General Insights

- **Quality vs. Performance:** Advanced methods (Canny, K-Means, IIR filters) improve quality but increase computation.
- **Noise vs. Detail:** Strong filtering improves clarity but may blur details; morphological operations can remove small artifacts while preserving shape.
- **Method Selection:** Depends on application requirements: edge detection for boundaries, region-based segmentation for classification, frequency filtering for enhancement.

Applications

- Medical imaging (noise removal, contrast enhancement, edge detection)
- Remote sensing and satellite imagery (segmentation, smoothing, feature extraction)
- Industrial inspection (defect detection, object recognition)
- Photography and computer vision (zooming, noise reduction, sharpening)

Conclusion

The techniques used together form a strong set of tools for studying and improving images. They show how methods from spatial processing, frequency analysis, morphology, and segmentation can work hand-in-hand to clean, enhance, and interpret visual data. Every method offers its own benefits and limitations, so it's important to choose the one that fits the image type, the kind of noise present, and the final goal. By using a mix of region-based and edge-based segmentation, noise reduction techniques, and frequency-domain enhancements, images can be processed in a way that makes them suitable for real-world tasks in areas like medical imaging, industrial inspection, and remote sensing.